

Before DSA:

If you are new to programming, Learn Fundamentals of Programming and solve Star Patterns.

DSA:

1. Neetcode 150: <https://neetcode.io/practice/practice/neetcode150>
2. Striver A2Z: <https://takeuforward.org/prep-hub/strivers-a2z-dsa-sheet>

System Design:

1. HLD:
<https://youtube.com/playlist?list=PLMCXHnjXnTnvo6aISjVkgxV-VH6EPyvoX&si=jtgLap5lGLHbv4J>
2. LLD:
https://youtube.com/playlist?list=PLTCrU9sGybupCpY20eked6blbHI4zZ55k&si=un_7i4xA7X27AUjC
3. check interview questions for HLD
<https://youtube.com/playlist?list=PL5q3E8eRUieWtYLmRU3z94-vGRcwKr9tM&si=fGXEuE9lhnmQntO>
4. check interview questions for
LLD:<https://youtube.com/playlist?list=PLYPO3T7Sl63u7uLLpiKCMXnRjeFIhUAvk&si=RaqTO1DAJIJRacFB>

Optional but important:

1. Abhishek veeramalla:
<https://youtube.com/playlist?list=PLdpzxOOAlwvIBIRWcReRV-m2kgIW6V6gr&si=6VJPVMd5FMGJAfvT>

Comp. Science Fundamentals:

Since you are already covering DSA (NeetCode), HLD (Gaurav Sen), and LLD (sudoCode), you have the core framework set up. Big Tech / FAANG interviewers rarely expect deep theoretical proofs for CS fundamentals unless you apply for specialized systems roles. However, they **do test applied CS concepts** in LLD, HLD, concurrency, and behavioral/technical drill-downs.

Focus on the exact topics to cover for **OS, DBMS & SQL**, and **CN**, trimmed to high-yield interview requirements.

1. Operating Systems (OS)

FAANG interviewers test OS concepts primarily to assess your ability to write **thread-safe code (LLD)** and build **scalable distributed systems (HLD)**.

- **Processes vs. Threads:**
 - Process state lifecycle, Process Control Block (PCB).
 - Threads sharing memory vs. process memory isolation.
 - Context switching cost (register saving, CPU cache invalidation).
 - User space vs. Kernel space, System Calls, `fork()`, `exec()`, Zombie and Orphan processes.
- **Concurrency & Synchronization (Crucial for LLD):**
 - Critical Section problem and Race Conditions.
 - **Mutex vs. Counting Semaphore vs. Binary Semaphore.**
 - Spinlocks, Reentrant Locks, Readers-Writers Lock, Condition Variables.
 - **Deadlocks:** 4 Necessary Conditions (Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait) and strategies to prevent/detect them.
 - Livelock vs. Starvation.
- **Memory Management:**
 - Paging, Page Tables, Page Faults, and Translation Lookaside Buffer (TLB).
 - Virtual Memory, Thrashing, and Page Replacement Algorithms (LRU, FIFO).
 - Cache locality (Spatial vs. Temporal) and L1/L2/L3 cache line implications.

2. DBMS & SQL

Big Tech focuses heavily on **transactions, indexing internals, and real SQL writing**, as these directly affect database design in HLD.

- **Transactions & Concurrency:**
 - **ACID Properties** in detail (Atomicity, Consistency, Isolation, Durability).
 - **4 Isolation Levels:** Read Uncommitted, Read Committed, Repeatable Read, Serializable.
 - **Read Anomalies:** Dirty Reads, Non-Repeatable Reads, Phantom Reads.
 - Lock-based concurrency (2-Phase Locking / 2PL) vs. Optimistic Concurrency Control (OCC) vs. MVCC (Multi-Version Concurrency Control).

- **Indexing & Storage Internals (Crucial for HLD):**
 - **B-Trees vs. B+ Trees** (Why B+ Trees are used for disk index storage).
 - **Clustered Index vs. Non-Clustered Index** (Heap vs. Primary Key ordering).
 - Composite Indexes and Index Cardinality.
 - Write-Ahead Logging (WAL) and how Durability is guaranteed.
 - Database Normalization (1NF, 2NF, 3NF, BCNF) vs. Denormalization (when to use in HLD).
- **SQL Query Writing (Practice ~30 LeetCode SQL Mediums):**
 - Aggregations (**GROUP BY**, **HAVING**).
 - All JOIN types (**INNER**, **LEFT**, **RIGHT**, **FULL**, **CROSS**, **SELF**).
 - **Window Functions** (**ROW_NUMBER()**, **RANK()**, **DENSE_RANK()**, **LEAD()**, **LAG()**, **OVER (PARTITION BY ...)**).
 - Subqueries vs. CTEs (**WITH** clauses).

3. Computer Networks (CN)

Interviewers test network protocols because every microservice in HLD communicates over the network.

- **Transport & Network Layers:**
 - OSI Model vs. TCP/IP Model (matching layers to protocols).
 - **TCP Mechanics:** 3-Way Handshake (**SYN**, **SYN-ACK**, **ACK**), 4-Way Teardown, Flow Control (Sliding Window), Congestion Control.
 - **TCP vs. UDP:** Detailed trade-offs (Reliability vs. Latency, use cases like Streaming/Gaming vs. Financial Transactions).
- **Application Layer & Web Protocols:**
 - **HTTP Evolution:** HTTP/1.1 (Keep-Alive), HTTP/2 (Multiplexing, Head-of-Line Blocking fix), HTTP/3 (QUIC over UDP).
 - **HTTPS & Security:** TLS/SSL Handshake, Symmetric vs. Asymmetric Encryption, PKI, Digital Certificates.
 - **API Protocols:** REST vs. WebSockets vs. gRPC (Protocol Buffers) vs. Long Polling.
- **Network Infrastructure (Crucial for HLD):**
 - **DNS Resolution Flow:** Root servers, TLD servers, Authoritative servers, Recursive resolvers, TTL.
 - **Layer 4 vs. Layer 7 Load Balancing.**
 - What happens under the hood when you enter [\[https://google.com\]](https://google.com)(<https://google.com>) into a browser address bar?

Quick Implementation Plan

1. **Targeted Study:** Allocate **3–4 hours a week** to CS theory while continuing DSA and System Design.
- 2.
3. **Focus on "Why" and "Trade-offs":** For example, don't just learn *what* a B+ Tree is—understand *why* DBMS engines prefer B+ Trees over Hash Indexes for range queries.
- 4.

5. **Practice Out-Loud Answers:** For classic questions (e.g., "*Explain Mutex vs. Semaphore*" or "*What happens when you type a URL in a browser?*"), practice delivering crisp 2-minute answers.